

Техническое задание

Система контроля версий с ASP.NET Core API, EF Core, реальным Git-движком и WPF-клиентом

Параметр	Значение
Версия документа	1.1
Дата	22.06.2026
Статус	Черновик для согласования перед разработкой
Ключевые решения	API + Swagger, WPF-клиент, EF Core + SQL Server LocalDB, LibGit2Sharp, Git через API, взаимные подписки

1. Назначение и границы проекта

Проект представляет собой упрощенную систему контроля версий, похожую по назначению на GitHub, но с меньшим набором функций. Система должна иметь серверное API, базу данных, авторизацию пользователей, подписки между пользователями, проекты, ветки и коммиты. Основной пользовательский интерфейс - десктопное WPF-приложение. Браузерный интерфейс ограничивается заглушкой.

- Сервер отвечает за учетные записи, подписки, ленту, права доступа, метаданные проектов и выполнение Git-операций.
- База данных хранит прикладные данные и метаданные; содержимое репозитория хранится в настоящих Git-репозиториях.
- Клиент WPF работает с сервером через HTTP API и не подключается к Git remote напрямую.
- Swagger должен быть доступен для проверки и ручного тестирования API.

2. Принятые архитектурные решения

Область	Решение	Обоснование
Сервер	ASP.NET Core Web API	Подходит для REST API, авторизации, Swagger и слоистой архитектуры.
База данных	EF Core + SQL Server LocalDB	Удобно для локальной разработки под Windows; миграции управляются EF Core.
Аккаунты	ASP.NET Core Identity + JWT	Identity закрывает регистрацию, парольные хэши и пользователей; JWT удобен для WPF-клиента.
Git	LibGit2Sharp	Реальный Git-движок внутри .NET-приложения.
Синхронизация	Git через API	WPF вызывает endpoint-ы; сервер проверяет права и выполняет операции через Git-слой.
Социальная модель	Взаимные подписки	Подписка односторонняя; если второй пользователь подписался в ответ, связь становится взаимной без отдельного подтверждения.
Клиент	WPF + MVVM	Соответствует требованию десктопного клиента и разделяет UI и логику.
Документация API	Swagger / Swashbuckle	Позволяет просматривать и тестировать endpoint-ы сервера.

3. Предполагаемая структура решения

```
VersionControlSystem/  
  VersionControlSystem.sln  
  src/  
    Vcs.Api/  
      Controllers/  
      Program.cs  
      appsettings.json  
      wwwroot/index.html  
    Vcs.Domain/  
      Entities/  
      Enums/  
    Vcs.Application/  
      DTOs/  
      Interfaces/  
      Services/  
      Validation/
```

```
Vcs.Infrastructure/  
  Data/  
  Migrations/  
  Auth/  
  Git/  
  Storage/  
  Repositories/  
Vcs.Desktop/  
  Views/  
  ViewModels/  
  Models/  
  Services/  
  Commands/  
tests/  
  Vcs.Api.Tests/  
  Vcs.Application.Tests/
```

Domain содержит сущности и enum-ы, Application - сценарии и DTO, Infrastructure - EF Core, Identity и Git-реализацию, Api - HTTP-контроллеры, Desktop - WPF-клиент.

4. MVP

4.1. Входит в MVP

- Регистрация, вход, получение текущего пользователя, выход на стороне клиента.
- Поиск пользователей, подписка на пользователя, отписка, список подписок и подписчиков.
- Подписка в ответ: второй пользователь может подписаться на первого, формируя взаимную связь.
- Лента публичных проектов пользователей, на которых подписан текущий пользователь.
- Создание проекта с типом доступа: публичный или приватный.
- Просмотр своих проектов, публичных проектов и проектов из ленты.
- Добавление участников в приватный проект и проверка доступа.
- Создание bare Git-репозитория на сервере при создании проекта.
- Получение списка веток, создание ветки, удаление ветки, выбор активной ветки.
- Создание коммита через API на основе переданных файловых изменений.
- Просмотр истории коммитов, деталей коммита, файлов проекта и diff.
- WPF-экраны: вход, регистрация, проекты, лента, подписки, проект, ветки, коммиты, просмотр diff.
- Swagger UI для ручного тестирования API.
- Статическая веб-заглушка на корневом маршруте сервера.

4.2. Не входит в MVP

- Полноценный Git-over-HTTP или SSH remote.
- Pull request, code review и inline-комментарии к коду.
- Merge conflict UI и сложное разрешение конфликтов.
- CI/CD, Actions, package registry, issues, wiki.
- Почтовая верификация и восстановление пароля, если не будет отдельного требования.

- Мобильный или полноценный браузерный интерфейс.

5. Функциональные требования

Код	Требование	Критерий приемки
FR-01	Пользователь может зарегистрироваться по email, username и паролю.	После регистрации пользователь существует в БД и может войти.
FR-02	Пользователь может войти и получить JWT access token.	Защищенные endpoint-ы принимают токен и определяют пользователя.
FR-03	Пользователь может искать других пользователей.	API возвращает пользователей по части username/email без раскрытия лишних данных.
FR-04	Пользователь может подписаться на другого пользователя.	В БД создается односторонняя подписка follower -> following.
FR-05	Пользователь может отписаться.	Подписка удаляется, проекты этого пользователя больше не появляются в ленте.
FR-06	Пользователь может подписаться в ответ.	В БД появляется обратная подписка; связь считается взаимной при наличии двух записей.
FR-07	Пользователь видит ленту публичных проектов тех, на кого он подписан.	API возвращает только публичные проекты followed-пользователей.
FR-08	Пользователь может создать публичный или приватный проект.	В БД создается проект, на диске создается bare Git-репозиторий.
FR-09	Публичный проект доступен для просмотра всем авторизованным пользователям.	API разрешает чтение проекта без членства.
FR-10	Приватный проект доступен только владельцу и участникам.	API возвращает 403 при отсутствии доступа.
FR-11	Владелец может добавлять участников проекта.	Участник получает доступ к приватному проекту.
FR-12	Пользователь с правами записи может управлять ветками.	Операции отражают реальное состояние Git.
FR-13	Пользователь с правами записи может создать коммит через API.	В Git появляется настоящий commit с автором, датой, сообщением и tree.
FR-14	Пользователь может посмотреть историю коммитов и diff.	API возвращает commit hash, автора, дату, сообщение и изменения.
FR-15	WPF-клиент должен покрывать основные сценарии MVP.	Сценарии выполняются без ручного обращения к Swagger.
FR-16	Swagger должен отображать все публичные API endpoint-ы.	При запуске сервера доступен Swagger UI.

6. Нефункциональные требования

Код	Требование
NFR-01	Пароли не хранятся в открытом виде; используется ASP.NET Core Identity.
NFR-02	Все операции с проектами, ветками и коммитами проверяют права доступа на сервере.
NFR-03	API возвращает предсказуемые HTTP-коды: 200, 201, 400, 401, 403, 404, 409, 500.
NFR-04	Репозитории хранятся вне папки wwwroot и недоступны как статические файлы.
NFR-05	Лента не должна раскрывать приватные проекты пользователей, на которых оформлена подписка.
NFR-06	Размер загружаемых файлов и пакетов изменений ограничивается настройкой сервера.
NFR-07	Код разделяется на слои; контроллеры не содержат прямой Git- и EF-логики.
NFR-08	Для ключевых сервисов должны быть подготовлены unit- или integration-тесты.

7. Модель данных

Сущность	Основные поля	Назначение
ApplicationUser	Id, UserName, Email, CreatedAt	Пользователь Identity.
UserSubscription	Id, FollowerId, FollowingId, CreatedAt	Односторонняя подписка пользователя на пользователя.
Project	Id, OwnerId, Name, Description, Visibility, RepositoryPath, DefaultBranch, CreatedAt, UpdatedAt	Метаданные проекта и путь к bare-репозиторию.
ProjectMember	Id, ProjectId, UserId, Role, CreatedAt	Участник проекта и минимальная роль доступа.
RefreshToken	Id, UserId, TokenHash, ExpiresAt, RevokedAt	Сессия обновления токена, если refresh token будет включен в первую реализацию.
AuditLog	Id, UserId, Action, EntityType, EntityId, CreatedAt	Опционально для журналирования важных действий.

Git-объекты - commit, tree, blob, branch - не дублируются в БД как источник истины. БД хранит только данные приложения: пользователей, подписки, права доступа, проекты и путь к репозиториям.

7.1. Связи

- ApplicationUser 1:N Project через OwnerId.
- ApplicationUser N:M Project через ProjectMember.
- ApplicationUser N:M ApplicationUser через UserSubscription: FollowerId -> FollowingId.
- Взаимная подписка определяется наличием двух записей: A -> B и B -> A.
- Project 1:1 Git repository через RepositoryPath на диске.

7.2. Сущности БД в виде кода

Ниже приведен базовый код сущностей для EF Core. Он фиксирует форму таблиц, связей и enum-ов для первой реализации. В проекте Identity рекомендуется использовать Guid-ключи: IdentityUser, IdentityRole и IdentityDbContext.

```
public enum ProjectVisibility
{
    Public = 1,
    Private = 2
}

public enum ProjectRole
{
    Owner = 1,
    Reader = 2,
    Writer = 3
}

public enum AuditActionType
{
    ProjectCreated = 1,
    ProjectUpdated = 2,
    MemberAdded = 3,
    BranchCreated = 4,
    CommitCreated = 5,
```

```

        UserSubscribed = 6,
        UserUnsubscribed = 7
    }

    public sealed class ApplicationUser : IdentityUser<Guid>
    {
        public DateTime CreatedAtUtc { get; set; } = DateTime.UtcNow;

        public ICollection<Project> OwnedProjects { get; set; } = [];
        public ICollection<ProjectMember> ProjectMemberships { get; set; } = [];

        public ICollection<UserSubscription> Following { get; set; } = [];
        public ICollection<UserSubscription> Followers { get; set; } = [];
    }

    public sealed class UserSubscription
    {
        public Guid Id { get; set; }
        public Guid FollowerId { get; set; }
        public Guid FollowingId { get; set; }
        public DateTime CreatedAtUtc { get; set; } = DateTime.UtcNow;

        public ApplicationUser Follower { get; set; } = null!;
        public ApplicationUser Following { get; set; } = null!;
    }

    public sealed class Project
    {
        public Guid Id { get; set; }
        public Guid OwnerId { get; set; }
        public string Name { get; set; } = string.Empty;
        public string? Description { get; set; }
        public ProjectVisibility Visibility { get; set; }
        public string RepositoryPath { get; set; } = string.Empty;
        public string DefaultBranch { get; set; } = "main";
        public DateTime CreatedAtUtc { get; set; } = DateTime.UtcNow;
        public DateTime? UpdatedAtUtc { get; set; }

        public ApplicationUser Owner { get; set; } = null!;
        public ICollection<ProjectMember> Members { get; set; } = [];
    }

    public sealed class ProjectMember
    {
        public Guid Id { get; set; }
        public Guid ProjectId { get; set; }
        public Guid UserId { get; set; }
        public ProjectRole Role { get; set; }
        public DateTime CreatedAtUtc { get; set; } = DateTime.UtcNow;

        public Project Project { get; set; } = null!;
        public ApplicationUser User { get; set; } = null!;
    }

    public sealed class RefreshToken

```

```

{
    public Guid Id { get; set; }
    public Guid UserId { get; set; }
    public string TokenHash { get; set; } = string.Empty;
    public DateTime ExpiresAtUtc { get; set; }
    public DateTime? RevokedAtUtc { get; set; }
    public DateTime CreatedAtUtc { get; set; } = DateTime.UtcNow;

    public ApplicationUser User { get; set; } = null!;
}

public sealed class AuditLog
{
    public Guid Id { get; set; }
    public Guid? UserId { get; set; }
    public AuditActionType Action { get; set; }
    public string EntityType { get; set; } = string.Empty;
    public Guid? EntityId { get; set; }
    public DateTime CreatedAtUtc { get; set; } = DateTime.UtcNow;

    public ApplicationUser? User { get; set; }
}

public sealed class AppDbContext
    : IdentityDbContext<ApplicationUser, IdentityRole<Guid>, Guid>
{
    public DbSet<UserSubscription> UserSubscriptions => Set<UserSubscription>();
    public DbSet<Project> Projects => Set<Project>();
    public DbSet<ProjectMember> ProjectMembers => Set<ProjectMember>();
    public DbSet<RefreshToken> RefreshTokens => Set<RefreshToken>();
    public DbSet<AuditLog> AuditLogs => Set<AuditLog>();

    public AppDbContext(DbContextOptions<AppDbContext> options)
        : base(options)
    {
    }

    protected override void OnModelCreating(ModelBuilder builder)
    {
        base.OnModelCreating(builder);

        builder.Entity<UserSubscription>()
            .HasIndex(x => new { x.FollowerId, x.FollowingId })
            .IsUnique();

        builder.Entity<UserSubscription>()
            .HasOne(x => x.Follower)
            .WithMany(x => x.Following)
            .HasForeignKey(x => x.FollowerId)
            .OnDelete(DeleteBehavior.Restrict);

        builder.Entity<UserSubscription>()
            .HasOne(x => x.Following)
            .WithMany(x => x.Followers)

```

```

        .HasForeignKey(x => x.FollowingId)
        .onDelete(DeleteBehavior.Restrict);

builder.Entity<Project>()
    .HasIndex(x => new { x.OwnerId, x.Name })
    .IsUnique();

builder.Entity<ProjectMember>()
    .HasIndex(x => new { x.ProjectId, x.UserId })
    .IsUnique();

builder.Entity<RefreshToken>()
    .HasIndex(x => x.TokenHash)
    .IsUnique();
    }
}

```

8. Хранение Git-репозиториев

Для каждого проекта сервер создает отдельный bare Git-репозиторий. Путь хранится в таблице Project.

```

storage/
  repositories/
    {projectId}.git/
  workspaces/
    {operationId}/
  temp/
  uploads/

```

- Bare-репозитории являются постоянным хранилищем Git-данных.
- Временные рабочие директории используются для создания коммитов, diff и операций, которым требуется checkout.
- После успешной операции временные директории очищаются.
- RepositoryPath не должен формироваться из пользовательского имени проекта; используется внутренний Guid/Id.

9. Поток создания коммита через API

Шаг	Описание
1	WPF-клиент отправляет projectId, branch, commit message и набор файловых изменений.
2	API проверяет JWT и права записи пользователя в проект.
3	Git-сервис создает временную рабочую директорию из нужной ветки bare-репозитория.
4	Сервер применяет изменения файлов: добавить, изменить, удалить.
5	LibGit2Sharp индексирует изменения и создает commit с автором текущего пользователя.
6	Указатель ветки обновляется на новый commit.
7	Временная директория удаляется, API возвращает hash нового коммита.

10. API endpoint-ы

Метод	Маршрут	Назначение	Доступ
POST	/api/auth/register	Регистрация пользователя	Anonymous
POST	/api/auth/login	Вход и выдача JWT	Anonymous
POST	/api/auth/refresh	Обновление access token	Authorized
GET	/api/auth/me	Текущий пользователь	Authorized
GET	/api/users/search?query=	Поиск пользователей	Authorized
POST	/api/subscriptions/{userId}	Подписаться на пользователя	Authorized
DELETE	/api/subscriptions/{userId}	Отписаться от пользователя	Authorized
GET	/api/subscriptions/following	Пользователи, на которых подписан текущий пользователь	Authorized
GET	/api/subscriptions/followers	Подписчики текущего пользователя	Authorized
GET	/api/subscriptions/mutual	Взаимные подписки	Authorized
GET	/api/feed/projects	Лента публичных проектов по подпискам	Authorized
POST	/api/projects	Создать проект и Git-репозиторий	Authorized
GET	/api/projects/my	Мои проекты	Authorized
GET	/api/projects/public	Публичные проекты	Authorized
GET	/api/projects/{id}	Детали проекта	Read access
PATCH	/api/projects/{id}	Изменить описание/видимость	Owner
DELETE	/api/projects/{id}	Удалить проект	Owner
POST	/api/projects/{id}/members	Добавить участника	Owner
GET	/api/projects/{id}/members	Список участников	Read access
DELETE	/api/projects/{id}/members/{userId}	Удалить участника	Owner
GET	/api/projects/{id}/branches	Список веток Git	Read access
POST	/api/projects/{id}/branches	Создать ветку	Write access
PATCH	/api/projects/{id}/branches/{name}	Переименовать ветку	Write access
DELETE	/api/projects/{id}/branches/{name}	Удалить ветку	Write access
GET	/api/projects/{id}/commits?branch=	История коммитов	Read access
GET	/api/projects/{id}/commits/{sha}	Детали коммита	Read access
POST	/api/projects/{id}/commits	Создать commit через Git-сервис	Write access
GET	/api/projects/{id}/files?branch=&path=	Файлы/каталог в ветке	Read access
GET	/api/projects/{id}/diff?from=&to=	Diff между коммитами	Read access

11. WPF-клиент

Экран	Назначение
LoginView	Вход пользователя, сохранение access token в клиентском состоянии.
RegisterView	Регистрация нового пользователя.
Shell/MainWindow	Основной контейнер, навигация между разделами.
ProjectsView	Список моих проектов, публичных проектов, создание проекта.
FeedView	Лента публичных проектов пользователей, на которых оформлена подписка.
SubscriptionsView	Поиск пользователей, подписаться, отписаться, подписчики, подписки, взаимные подписки.
ProjectDetailsView	Информация о проекте, участники, выбранная ветка.

Экран	Назначение
BranchesView	Список, создание и удаление веток.
CommitsView	История коммитов выбранной ветки.
CommitDetailsView	Детали коммита, файлы и diff.

Клиентская архитектура строится по MVVM: View содержит только разметку, ViewModel управляет состоянием и командами, Services выполняют HTTP-запросы и локальную подготовку файлов.

12. Безопасность и права доступа

- Все endpoint-ы, кроме регистрации, входа, Swagger в dev-режиме и веб-заглушки, требуют авторизации.
- Подписка не дает доступ к приватным проектам сама по себе.
- Лента показывает только публичные проекты пользователей, на которых подписан текущий пользователь.
- Публичный проект доступен на чтение авторизованным пользователям.
- Приватный проект доступен владельцу и участникам.
- Операции записи доступны владельцу и участникам с ролью записи. Минимальная ролевая модель должна быть подтверждена перед реализацией.
- Сервер не доверяет данным клиента о пользователе, владельце проекта или правах.
- Пути к файлам внутри коммита должны нормализоваться, чтобы запретить выход за пределы рабочей директории.

13. План реализации

Этап	Результат
1. Каркас решения	Созданы проекты, подключены зависимости, настроена слоистая структура.
2. API-инфраструктура	Program.cs, Swagger, EF Core, LocalDB, миграции, Identity, JWT.
3. Пользователи	Регистрация, вход, текущий пользователь, базовые DTO и валидация.
4. Подписки и лента	Поиск пользователей, подписки, подписчики, взаимные подписки, лента публичных проектов.
5. Проекты и права	Создание проектов, публичность/приватность, участники, проверки доступа.
6. Git-сервис	Создание bare repo, ветки, история коммитов, файлы, diff.
7. Коммиты через API	Прием изменений, временный workspace, commit через LibGit2Sharp.
8. WPF MVP	Навигация, auth, проекты, лента, подписки, ветки, коммиты, diff.
9. Тестирование	Проверка auth, subscriptions, feed, access rules, Git-операций и основных API-сценариев.
10. Полировка	Обработка ошибок, UI-состояния, README, инструкции запуска.

14. Тестирование

- Unit-тесты Application-сервисов: права доступа, проекты, подписки, лента.
- Integration-тесты API: регистрация, вход, подписка, лента, создание проекта, приватный доступ.
- Git integration-тесты: создание репозитория, ветки, коммита, чтение истории и diff.
- Ручная проверка Swagger для всех endpoint-ов.
- Ручная проверка WPF-сценариев MVP.

15. Открытые вопросы перед разработкой

Вопрос	Почему важно
Ролевая модель проекта: только Owner/Member или Owner/Reader/Writer?	Влияет на БД, проверки доступа и UI управления участниками.
Можно ли добавлять в приватный проект любого пользователя или только взаимную подписку?	Влияет на сценарий приглашений и endpoint добавления участника.
Показывать ли собственные публичные проекты в ленте или только проекты тех, на кого подписан пользователь?	Влияет на логику /api/feed/projects и UX ленты.
Нужны ли refresh token в MVP?	Влияет на таблицу сессий и UX WPF-клиента.
Максимальный размер файла/коммита?	Нужно для лимитов API и защиты сервера.
Нужно ли хранить локальные рабочие копии в WPF или отправлять изменения из выбранной папки без постоянной копии?	Влияет на UX клиента и LocalGitService.

До ответа на эти вопросы разработку можно начинать с каркаса, авторизации, подписок, проектов и базового Git-сервиса, но финальную модель прав и UX коммитов лучше согласовать заранее.

16. Критерии готовности MVP

- Сервер запускается локально, подключается к SQL Server LocalDB и применяет миграции.
- Swagger отображает и позволяет проверить основные endpoint-ы.
- Пользователь может зарегистрироваться, войти и создать проект.
- Пользователь может подписаться на другого пользователя и увидеть его публичные проекты в ленте.
- Взаимная подписка определяется двумя односторонними подписками.
- При создании проекта создается настоящий bare Git-репозиторий.
- Через WPF можно создать проект, ветку и commit, затем увидеть commit в истории.
- Права доступа к приватному проекту проверяются сервером.
- Подписки и лента работают через API и отображаются в WPF.
- В репозитории есть README с инструкцией запуска сервера и WPF-клиента.